In this section, we discuss the advantages of NextLat compared to prior transformer learning approaches, focusing discussions on belief-learning methods, i.e., BST and JTP. We provide brief descriptions of these methods and their training objectives in Section A. Motivated readers interested in further details beyond the scope of this work may refer to Hu et al. [2025] and Ahn et al. [2025]. Table 4 summarizes the training and inference parameters, training speed (in iterations per second), and gradient signal characteristics for each method on FineWeb-Edu pretraining, providing context for the discussions that follow.

Computational Costs. While BST benefits from $O(T^2)$ gradient signals per token sequence, this is a double-edged sword. Even with the optimized implementation of Hu et al. [2025], training remains extremely costly because gradients must be accumulated over all $O(T^2)$ predictions of different prefix–suffix pairs. Moreover, BST trains two transformer encoders, further increasing compute and parameter cost. During inference, BST also uses both transformer encoders, one for generation and the other for scoring the likelihood of the generated sequence. On FineWeb-Edu pretraining, BST is over $3\times$ slower than NextLat in training speed (see Table 4). Note that this result already uses 10% subsampling of prefix-suffix pairs to prevent out-of-memory issues; the training speed of BST would be much slower if all pairs were used for training. In contrast, NextLat with $d = 1$ incurs negligible overhead relative to GPT while achieving the same belief-state learning guarantees as BST.

We next compare the compute costs of the multi-step prediction methods, i.e., MTP, JTP, and NextLat. The MTP implementation of Gloeckle et al. [2024], as well as other variants such as the one introduced in Liu et al. [2024], require additional transformer layers as the prediction horizon d increases, whereas JTP and NextLat keep parameter counts fixed across horizons. JTP and NextLat exhibit similar training speeds for $d = 1$[6], while MTP lags behind. At $d = 8$, JTP is substantially faster than NextLat because NextLat sequentially unrolls its latent dynamics model p_ψ to compute multi-step losses, while JTP computes them in parallel. Nonetheless, this modest increase in computation for NextLat yields substantially better performance than JTP across all benchmarks. Importantly, NextLat's sequential computation remains far more efficient than that of recurrent neural networks (RNN), which we discuss further in Section 5.2.

Belief State Learning. GPT and MTP lack any theoretical learning pressure to form belief-state representations, which means that they do not necessarily learn sufficient representations predictive of future observations. JTP can learn belief states but only under the restrictive condition that the prediction horizon satisfies $d \geq k$, where k denotes the observability horizon of the underlying data-generating process (see Theorem A.1 in appendix). In long-context sequence modeling, however, the underlying process is often k-observable for a very large k, rendering this condition impractical. NextLat, on the other hand, learns belief-state representations independently of d and larger multi-step prediction horizons are used only to provide richer gradient signals. It also avoids the expensive $O(T^2)$ gradient computation required by BST to learn belief states. As such, NextLat stands as a simple, computationally efficient, and theoretically grounded alternative to existing belief-state learning approaches (i.e., BST and JTP) in autoregressive sequence modeling.

Myopic Nature of Token-level Predictions. Token-level supervision is often myopic. Next-token prediction transformers tend to prioritize short-term dependencies, and studies have shown that early training can often resemble n-gram modeling, which can delay or even prevent learning of the true Markov kernel [Edelman et al., 2024, Makkuva et al., 2025]. Bachmann and Nagarajan [2024] further showed that the myopic nature of next-token prediction training can trap models in suboptimal local minima, undermining long-horizon planning. In our experiments, we find that adding additional token-level prediction objectives, as in BST, JTP, and MTP, not only degrades next-token performance but also fails to yield consistent gains in generalization. In contrast, NextLat, which emphasizes latent transition modeling as its primary objective, minimizes degradation in next-token performance and improves downstream generalization by encouraging the learning of predictive representations rather than shallow token-level correlations. Moreover, as discussed in Section 3.1, latent-space supervision provides denser gradient signals than token-level supervision. Overall, NextLat offers a compelling alternative to conventional token-level prediction objectives for transformer training.

[6]JTP exhibiting a slightly faster training speed than GPT at $d = 1$ is likely an artifact of `torch.compile()` due to differences in the computation graph.

5.2 NextLat vs. Recurrent Neural Networks

Algorithmic reasoning requires capabilities most naturally understood through recurrent models of computation, like the Turing machine. However, strict recurrence imposes a sequential computation bottleneck at training time. NextLat introduces a *recurrent inductive bias* for the learned representations via latent transition prediction without turning the transformer into a strictly sequential model.

Training RNNs without recurrence. Intuitively, NextLat ($d = 1$) can be viewed as *fully parallel* co-training of a transformer and a recurrent neural network (RNN). The transformer forecasts latent states across the sequence in parallel, while the latent dynamics model (the RNN) is trained using only one-step latent predictions. This co-training process does not require any sequential computation across time.

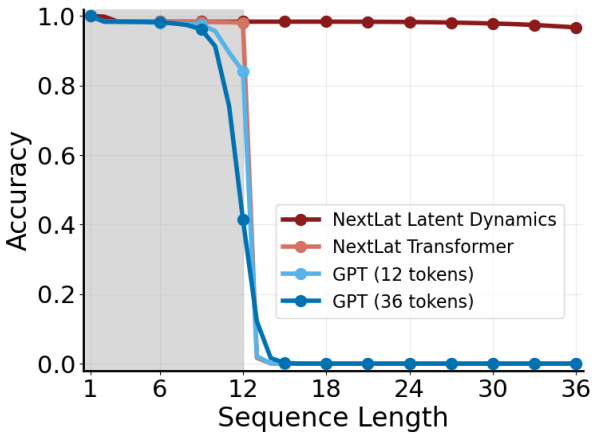


Figure 10: Length Generalization for A_5 word problem.

To study this interaction, we consider the A_5 word problem, a difficult *state-tracking* task involving even permutations of five elements. The problem is NC^1-complete and therefore inexpressible by constant depth, fixed-precision transformers, which are restricted to the $\text{TC}^0 \subset \text{NC}^1$ circuit complexity class [Merrill and Sabharwal, 2023][7]. We train 2-layer transformers using next-token prediction (GPT) and next-latent prediction (NextLat, $d = 1$) on the A_5 word problem with a sequence length of 12 tokens (permutations). We then tested the transformers and the latent dynamics model (RNN) on their length generalization to sequences of length 36. Note that during inference, the RNN only uses the transformer backbone to initialize the hidden state for the first token, after which subsequent hidden state transitions and token predictions are generated independently by the RNN without invoking the transformer. Importantly, the RNN also has substantially fewer parameters than the transformer (2.62M vs. 6.43M parameters).

Figure 10 shows interesting results. First, the transformer trained with **NextLat exhibits better state-tracking performance than GPT** within the 12-token training horizon. More surprisingly, while the transformer itself fails to generalize beyond the 12-token horizon, the learned latent dynamics model successfully generalizes to 36-token sequences ($> 95\%$ accuracy). This is particularly striking because even a GPT model trained directly on 36-token sequences ("GPT (36 tokens)") is unable to solve the task, consistent with previous results [Merrill et al., 2025]. In other words, although the transformer itself cannot solve the 36-token A_5 problem, it is nevertheless able to train an RNN that can.

From a circuit complexity perspective, solving problems in NC^1 requires transformers with $O(\log T)$ depth [Merrill and Sabharwal, 2023, Liu et al., 2023], where T is the sequence length. This explains why the shallow 2-layer transformers (both GPT and NextLat) are unable to solve problems beyond 12 tokens. What is surprising, however, is that under NextLat training, **the co-trained RNN can generalize far beyond the expressivity limits of the underlying transformer, despite being trained entirely in parallel**. This raises an intriguing question:

> *Given inputs of length T, can a transformer with $O(\log T)$ depth co-train (in parallel) an RNN that generalizes to sequences of length $\gg T$? More specifically, can the co-trained RNN learn NC^1 computations, even though it is supervised using representations from a transformer in TC^0?*

Intuitively, this seems plausible. As long as the transformer has sufficient depth to express and forecast hidden states within the T-length training context window, it makes sense for the latent dynamics model (RNN) to learn reusable computational circuits that extrapolate far beyond it, as empirically suggested by Figure 10. In this view, NextLat may offer a mechanism for partially escaping the practical limitations imposed by the *parallelism tradeoff* [Merrill

[7] This assumes the widely held conjecture that $\text{TC}^0 \neq \text{NC}^1$.

and Sabharwal, 2023]: the transformer provides efficient parallel representation learning, while the RNN focuses on learning generalizable sequential update rules.

Computational Efficiency and Relation to Backpropagation Through Time. Now, we further compare NextLat against traditional RNNs trained using backpropagation through time. Tranditional RNNs incur $O(T)$ sequential dependence during training. In contrast, NextLat adds only an additional sequential cost proportional to the rollout horizon $d \ll T$, corresponding to unrolling the latent dynamics model p_ψ for d steps. This allows NextLat to largely retain the transformer's parallel training efficiency while also inheriting a recurrent inductive bias.

Conceptually, the one-step and multi-step prediction in NextLat resembles truncated backpropagation through time (TBPTT) in RNNs, with truncation windows of 1 and d, respectively. However, a key distinction lies in how gradients are propagated. In TBPTT for RNNs, gradient computation is truncated beyond the chosen window, yielding *biased* gradient estimates that lack convergence guarantees. In contrast, the **theoretical convergence of belief-state learning in NextLat, as shown in Theorem 3.2, is irrespective of** d, since it requires only one-step prediction optimality. Intuitively, NextLat performs full backpropagation through the transformer's computation graph, where self-attention connects all past tokens. The latent dynamics model operates in an "outer loop", aligning the transformer's hidden states to be temporally consistent across steps. Crucially, this outer-loop supervision does not truncate gradient flow within the transformer, and therefore avoids bias with respect to d. Larger prediction horizons simply provide richer supervision and faster empirical convergence.

Expressivity of the Recurrence. Modern state-space models (SSMs), such as S4 and Mamba, implement efficient linear recurrent updates while remaining highly parallelizable [Gu et al., 2022, Gu and Dao, 2024]. In contrast to SSMs, the latent dynamics model p_ψ in NextLat can express nonlinear transitions. Moreover, NextLat does not explicitly perform recurrence in the forward computation. Instead, recurrent-like dynamics emerge implicitly through one-step or multi-step unrolling of the p_ψ and aligning successive hidden states via regression. This induces temporal consistency within the latent space without altering the transformer architecture. However, it is important to note that NextLat modifies the learned representations, not the underlying circuit complexity. The overall computational expressivity of the fixed-depth transformer trained with NextLat remains bounded by that of constant-depth threshold circuits, i.e., within the TC^0 complexity class [Merrill et al., 2022]. However, earlier results in Figure 10 question whether the co-trained latent dynamics model (RNN) is subject to the same TC^0 limitations.

Hybrids Attention Models. NextLat differs fundamentally from hybrid Transformer–SSM architectures [Park et al., 2024, Lieber et al., 2024, Ren et al., 2025], which explicitly interleave SSM and attention layers. NextLat requires no architectural changes and operates purely as an auxiliary training objective on the model's latent representations. As such, it is broadly compatible with diverse sequence-modeling architectures. In fact, NextLat could complement hybrid models by encouraging compression and consistency in the residual attention pathways that fall outside the SSM's recurrent structure, potentially improving representation efficiency and generalization.

6 Limitations and Future Work

While NextLat shows strong empirical performance, several limitations remain in our work. First, we do not explore the design space of the latent dynamics model; all experiments use simple MLPs to isolate and demonstrate the effectiveness of the core NextLat approach. More expressive architectures may further improve performance. We also do not study how the width of the hidden layers in the latent dynamics MLP affects learning, even though it effectively acts as a bottleneck that constrains belief-state capacity and may influence performance across tasks. Empirically, we observe that using smaller latent dimensions is beneficial on tasks such as Path-Star graph and Countdown.

Second, the design of the NextLat objective (e.g., stop-gradients, KL self-distillation, Smooth L1 loss) is guided largely by small-scale ablations in Section D and empirical intuition. It remains unclear whether multi-step supervision ($d > 1$) and KL token-level supervision are even necessary at larger model and data scales. More systematic studies are needed to better understand how these components interact. Third, due to computational constraints, we did not evaluate

against more recent or specialized MTP variants such as the one introduced in Liu et al. [2024]. Finally, we did not fully exploit the variable-length nature of NextLat's speculative decoding. In our experiments, the draft length remained fixed throughout decoding for each prompt; we only varied the static draft length between 2 and 10 tokens to identify the configuration with the highest inference speedup. We leave the exploration of more creative adaptive-length speculative decoding strategies for NextLat to future work.

On the analysis side, we do not study the structure of the learned representations under NextLat, leaving open questions about how the method shapes latent spaces. In Section E, we also highlight several quirks observed during pretraining with NextLat, such as increases in Smooth L1 loss ($\mathcal{L}_{\text{next-h}}$) during training and differing loss trajectories across optimizers. These observations suggest that NextLat can be sensitive to optimization dynamics. Better understanding of how to scale and parameterize the NextLat objective remains an important direction for future work.

This work represents only an preliminary study of next-latent prediction, leaving many promising directions for future research. Since the method requires no architectural changes beyond a lightweight latent dynamics model for shaping representations, an interesting direction is to apply it as a post-hoc finetuning objective for pretrained transformers. This could potentially improve reasoning, planning, and world-modeling capabilities of existing models without retraining from scratch. Moreover, because NextLat effectively organizes latent representations with recurrent-like dynamics, an interesting question is whether transformers trained with NextLat are better suited for RL post-training, where value estimation (be it implicit or explicit) benefits from such recursive "Bellman-like" latent structure. Finally, it would be valuable to explore richer latent architectures, such as higher-dimensional or hierarchical belief states spanning multiple layers or tokens, which may further improve long-horizon reasoning and planning.

7 Conclusion

In this paper, we introduced Next-Latent Prediction (NextLat), a simple yet powerful framework that augments next-token training with self-supervised latent-space prediction, enabling transformers to learn belief-state representations. Theoretically, we show that the pressure to form concise latent summaries of past information sufficient to predict the future arises naturally from the NextLat objective. Empirically, NextLat yields more compact, predictive, and generalizable representations across tasks in world modeling, reasoning, planning, and language modeling—all without changing the transformer's architecture, parallel training efficiency, or inference procedure. By reintroducing a recurrent inductive bias through self-predictive latent dynamics, NextLat unifies the inherent bias toward compact and temporally consistent representations of recurrent models with the scalability and parallelism of transformers. Our method is broadly applicable for training transformers in autoregressive sequence modeling domains. Looking ahead, we view NextLat as a step toward training objectives that endow autoregressive sequence models with simpler, more compact, and therefore more generalizable representations of complex systems.

Acknowledgements

We thank Jason Eisner, Jordan T. Ash, Pradeep Varakantham, Andrea Zanette, Michael C. Mozer, Ying Fan, and the MSR AI Frontiers team for their valuable discussions and support.

References

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Kwangjun Ahn, Alex Lamb, and John Langford. Efficient joint prediction of multiple future tokens, 2025. URL https://arxiv.org/abs/2503.21801.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son